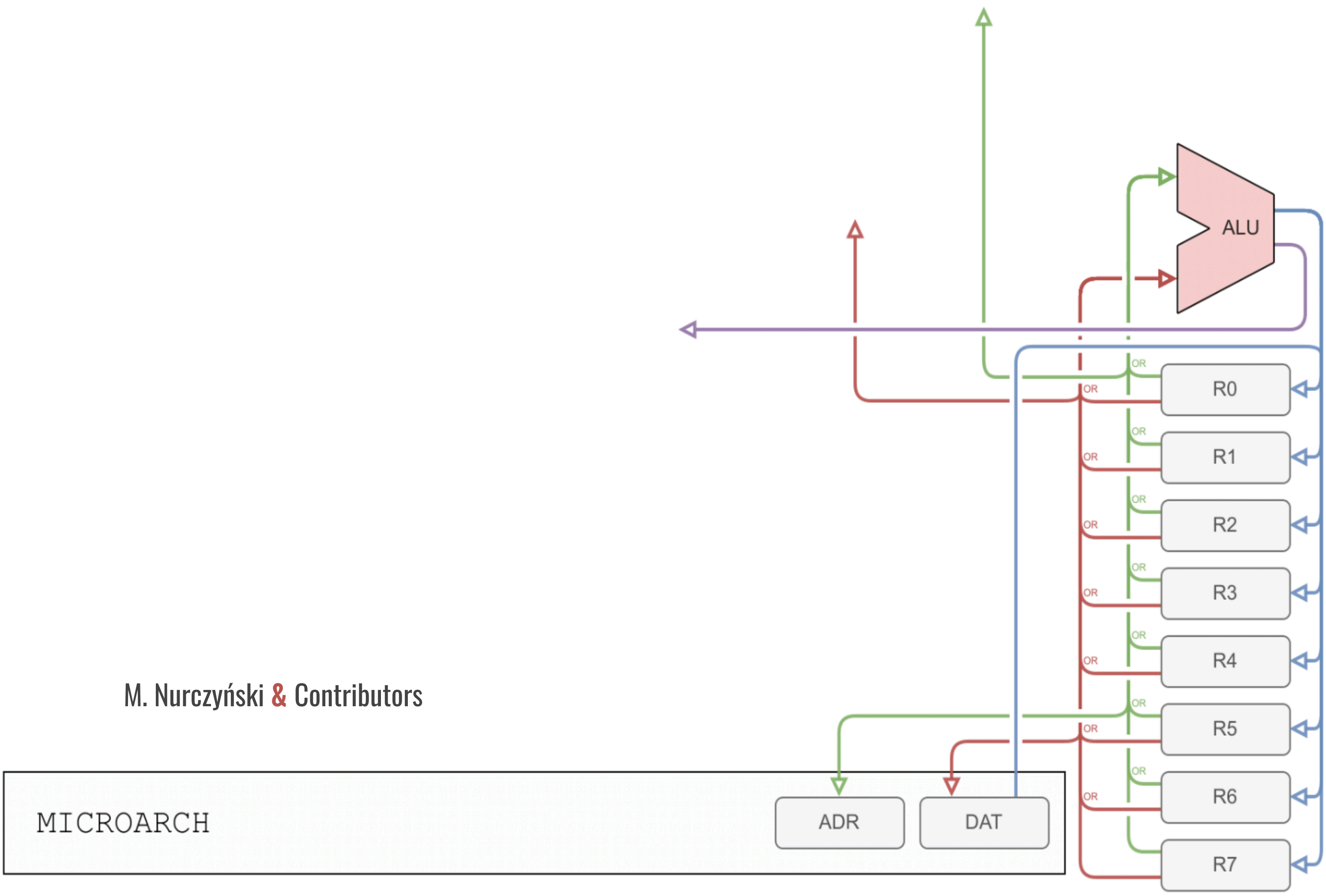
microarch

Instruction Set Architecture



M. Nurczyński & Contributors

Table of Contents

Overview

microarch is a minimalistic, 8-bit, extensible, RISC, Harvard, load-store computer architecture intended for small low-power (or power constrained) microcontrollers.

This document describes the assembly instructructions available to **microarch** programmers - their function, syntax, and encoding. Each instruction will be given its own section in this document. Some commonly expected information may be missing from this specification as it was designed to be implementation agnostic, as such it only defines the requirements and certain limits that all implementations must adhere to. To learn about those omitted here specifications consult the manual of your specific implementation.

---

microarch offers 8-bit data address space and up to 16-bit instruction address space.

Data memory has standard 1-byte memory cells, while the program memory has 3-byte cells (corresponding to the 3-byte long **microarch** instruction word).

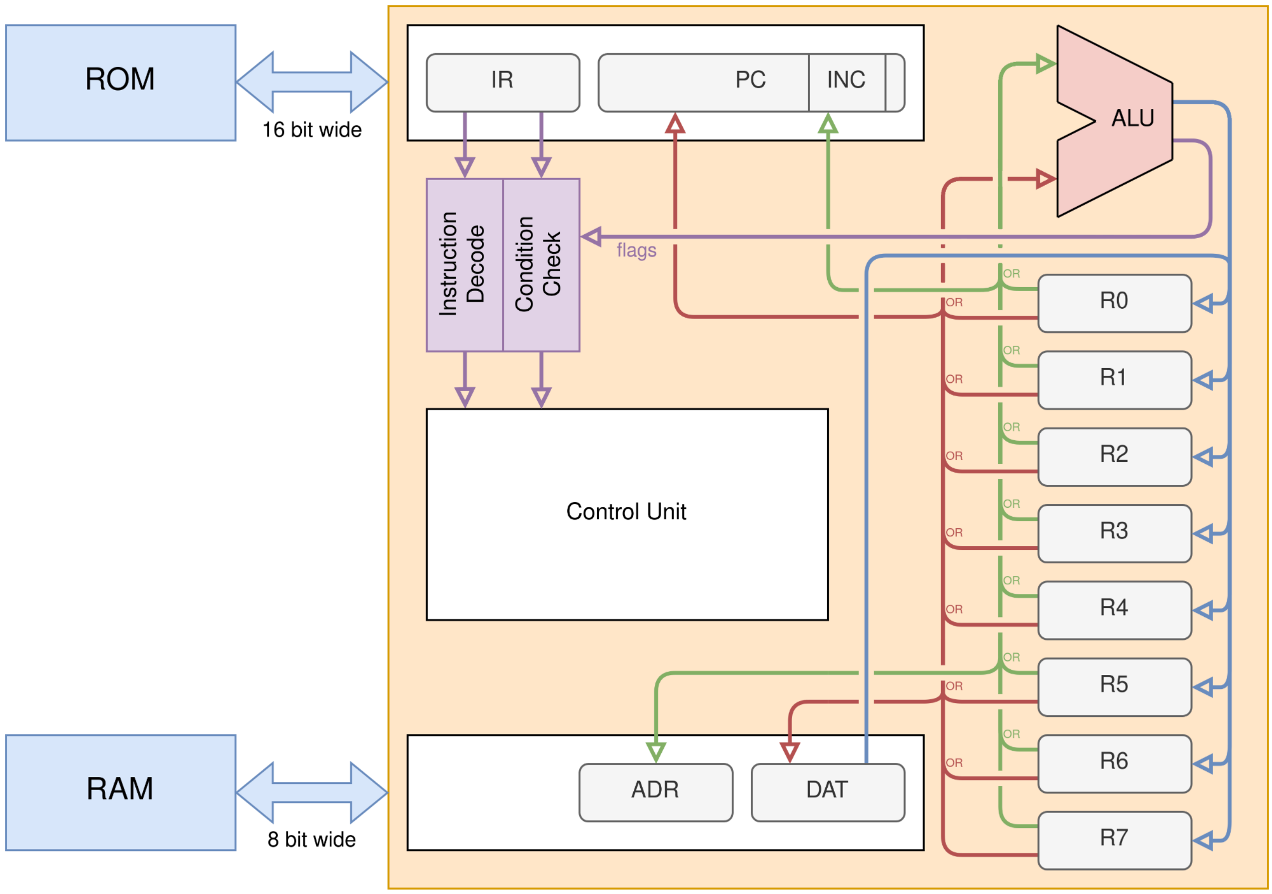
That allows for up to 256 bytes of directly addressable data memory and up to 192kB (3 times 64kB) of program memory, giving space for a maximum of 65536 instructions.

The microarch processor has 8 registers available to the programmer, numbered R0 through R7, with a powerful, unique register addressing system, that will be described later on in this document.

The architecture offers several power-saving stand-by modes that can be directly invoked by the program. It also offers a single interrupt line.

Block Diagram

Basic reference block diagram of a **microarch** core



Register Bank

There are 8 registers available to the programmer (numbered from R0 to R7).

We recommend using the R7 register as stack pointer, in programs making use of stack (since there is no hardware support for stack in **microarch**, this is not enforced in any way).

Registers in **microarch** processors are addressed through special 8 bit values, described below, that will be referred to as “registry sets”.

Registry sets

“registry set” is an 8-bit value (a byte) describing any selection of registers in the form of a bitmask:

R7	R6	R5	R4	R3	R2	R1	R0

The youngest bit references the R0 register, the second-youngest bit references the R1 register and the oldest bit references the R7 register. For example:

Byte 00001000 means register R4.
Byte 01101100 means registers R6, R5, R3, R2.
Byte 00000000 means no registers. When used as input for an instruction, the value should be 0.
Byte 11111111 means registers R7, R6, R5, R4, R3, R2, R1, R0 (so all registers).

Input registry sets

When a registry set is given as instruction input, the value used will be the result of a bitwise OR operation of all registers specified in the registry set.

For example: When R1 is set to 01110010 and R4 is set to 00110100, then the value of the registry set 00010010 will be 01110110.

Output registry sets

When a registry set is given as instruction output, the value will be placed into all registers specified by the registry set.

Instructions

Instructions in **microarch** have a 3-byte long instruction word. All instructions are also conditional (based on 2 flags set by the CMP instruction).

Encoding

Below is a general instruction encoding scheme for microarch instruction. Exceptions may apply, in which case encoding for a specific instruction will be specified in the instruction listing below.

Op-code	Co1	Co2	Argument-1	Argument-2
4 bit	2-bit	2-bit	8-bit	8-bit

Op-code - Instruction code. Code 1101 is reserved for future use.
Co-1 - Condition 1 (flag ZF)
Co-2 - Condition 2 (flag CF)

Conditions

Each instruction has 2, 2-bit long condition fields (one for the ZF flag and one for the CF flag - those flags are described in further detail in the CMP instruction listing). Condition field works as follows:

True-only bit (older bit)	False-only bit (younger bit)
If 1 - allow true flag value If 0 - forbid true flag value	If 1 - allow false flag value If 0 - forbid false flag value

Therefore:
If the condition is 00 - the instruction will never be executed.
If the condition is 01 - the instruction will be executed only if the flag is set to false.
If the condition is 10 - the instruction will be executed only if the flag is set to true.
If the condition is 11 - the instruction will be executed regardless of the state of the flag.

Instruction listing

Below are described in detail all standard **microarch** instructions.

Naming Convention
Every instruction shall be named using 3 letter mnemonics.

NOP (No operation)

Opcode	Argument 1	Argument 2
0000	Reserved, must be 0	Reserved, must be 0
Do nothing. Both arguments must be 0 for the encoding to be valid, therefore the op-code may be used for other instructions in ISA extensions.		

CMP (Compare registers)

Opcode	Argument 1	Argument 2
1111	Input/Output registry set	Input registry set
Subtract input Argument 2 from input Argument 1, write the result to output Argument 1, set the flag CF to the value of the carry bit from the operation, set the flag ZF to 0 if the subtraction result is not equal to 0, set the flag ZF to 1 if the subtraction result is equal to zero.		

ADD (Add registers)

Opcode	Argument 1	Argument 2
1110	Input/Output registry set	Input registry set
Add input Argument 2 to input Argument 1, write the result to output Argument 1		

SET (Set registers to an immediate)

<table>
<tr><th>Opcode</th><th>Argument 1</th><th>Argument 2</th></tr>
<tr><td>0001</td><td>Output registry set</td><td>8 bit immediate value</td></tr>
<tr><td colspan="3">Set output registry set (Argument 1) to the immediate value specified by the value in Argument 2.</td></tr>
</table>

MOV (Move between registers)

<table>
<tr><th>Opcode</th><th>Argument 1</th><th>Argument 2</th></tr>
<tr><td>1100</td><td>Output registry set</td><td>Input registry set</td></tr>
<tr><td colspan="3">Copy value from input registry set (Argument 2) into the output registry set (Argument 1).</td></tr>
</table>

LDM (Load memory to registers)

<table>
<tr><th>Opcode</th><th>Argument 1</th><th>Argument 2</th></tr>
<tr><td>0010</td><td>Output registry set</td><td>Input registry set</td></tr>
<tr><td colspan="3">Load byte from memory from address specified by the input registry set (Argument 2) into the output registry set (Argument 1).</td></tr>
</table>

STM (Store registers in memory)

Opcode	Argument 1	Argument 2
0011	Output registry set	Input registry set
Store byte into memory at the address specified by the output registry set (Argument 1) from the input registry set (Argument 2).		

NAD (Bitwise NAND)

Opcode	Argument 1	Argument 2
1000	Input/Output registry set	Input registry set
Read values from both input registry sets (Argument 1 & 2) bitwise NAND them together and save the result into the output registry set (Argument 1).		

AND (Bitwise AND)

Opcode	Argument 1	Argument 2
1001	Input/Output registry set	Input registry set
Read values from both input registry sets (Argument 1 & 2) bitwise AND them together and save into the result the output registry set (Argument 1).		

XOR (Bitwise XOR)

<table>
<tr><th>Opcode</th><th>Argument 1</th><th>Argument 2</th></tr>
<tr><td>1010</td><td>Input/Output registry set</td><td>Input registry set</td></tr>
<tr><td colspan="3">Read values from both input registry sets (Argument 1 & 2) bitwise XOR them together and save into the result the output registry set (Argument 1).</td></tr>
</table>

SHR (Shift N bits right)

<table>
<tr><th>Opcode</th><th>Argument 1</th><th>Argument 2</th></tr>
<tr><td>1011</td><td>Input/Output registry set</td><td>8 bit, Immediate value</td></tr>
<tr><td colspan="3">Shift value of an input/output registry set (Argument 1) by an offset given as an immediate value (Argument 2). Only the lower 3 bits of the offset are used, an implementation is allowed to not support offsets larger than 1.</td></tr>
</table>

CID (CPU Identification)

<table>
<tr><th>Opcode</th><th>Argument 1</th><th>Argument 2</th></tr>
<tr><td>0110</td><td>Reserved, must be 0</td><td>8 bit, Immediate value</td></tr>
<tr><td colspan="3">Return CPU information and identification. This function can be used to query the CPU for supported extensions and other details. The immediate value (Argument 2) specifies the page number to query. Registers R0 though R3 are modified and R4 through R7 are left as is. For full documentation see the Identification section.</td></tr>
</table>

JPR (Jump to address in registers)

<table>
<tr><th>Opcode</th><th>Argument 1</th><th>Argument 2</th></tr>
<tr><td>0100</td><td>Input registry set, older 8 bits</td><td>Input registry set, younger 8 bits</td></tr>
<tr><td colspan="3">The two input register values are combined to form a 16 bit value. The processor executes a jump to that value.</td></tr>
</table>

JPI (Jump to immediate address)

<table>
<tr><th>Opcode</th><th>Argument 1</th><th>Argument 2</th></tr>
<tr><td>0101</td><td>Immediate value, older 8 bits</td><td>Immediate value, younger 8 bits</td></tr>
<tr><td colspan="3">The two immediate values are combined to form a 16 bit value. The processor executes a jump to that value.</td></tr>
</table>

CTR (Control)

<table>
<tr><th>Opcode</th><th>Argument 1</th><th>Argument 2</th></tr>
<tr><td>0111</td><td>Input registry set.</td><td>Immediate value.</td></tr>
<tr><td colspan="3">This instruction changes the value of the CPU control register - CTR. Argument 1 specifies which registry set the value will be read from. Argument 2 is a bitmask specifying what bits of the control register are changed. A given bit from the registry set specified by Argument 1 will be written to CTR, only if the corresponding bit from Argument 2 is set to 1.
Example: Argument 1 is set to 00010000, Register R4 is set to 00001111, CTR is set to 00010110, Argument 2 is set to 00011000. CTR will be set to 00001110.
This system allows for changing only select bits of the CTR register, without knowing values of other bits.
For details on the CTR register see the Control Register section of this document.</td></tr>
</table>

Memory Map

In the following chapter of this document we specify certain fixed memory addresses, for both data memory and instruction memory, that a **microarch** processor expects will be used for specific applications.

Memory banks

For supporting data memory sizes larger than 256 bytes, memory banks can be utilized.

Current memory bank number should be placed at memory address 0x03, whenever applicable, and synchronized across all banks.

Interrupts

The **microarch** processor core supports a single interrupt line. When an interrupt is received, the processor will finish execution of the current instruction. Then it will store the return address, in Big Endian at addresses 0x00 and 0x01 in the data memory and execute a jump to address 0x0001 in the instruction memory, where the interrupt handler should be implemented. Please note that every address in the program memory represents 3 bytes (1 instruction). An interrupt automatically switches off interrupts and resumes execution, if it was stopped (see the **Control Register** section for details).

If the processor allows more than 1 interrupt source, an 8-bit interrupt number should be stored under address 0x02 in data memory. Otherwise the value of that byte after an interrupt is undefined.

If the implementation supports memory banks, those fields should be placed in memory bank 0.

I/O Devices and Interfaces

Control registers for I/O interfaces (both parallel and serial ports) and other devices (for example a system for writing into the microcontroller's flash memory) should be mounted at memory addresses from 0x04 to 0x1F.

If the implementation supports memory banks, those fields should be placed in memory bank 0.

CPU Identification

For each valid CID page number (shown in the first column) the CID instruction reads up to 32 bits of CPU specific data. The table below shows the meaning of particular bits. Extension bits tell if a specific extension is supported. Currency there are no registered extensions.

Page Number	R0	R1	R2	R3
0	Reserved for future use	Extension bits, block A	Extension bits, block B	Extension bits, block C

Control register

The Control Register is an 8 bit register that determines certain settings of the CPU.

Bits	7 - 5	4	3 - 0
Name	ES	IF	Reserved for ISA extensions

ES - Execution status. Allowed values from 0 to 7. Value 0 corresponds to normal execution. Setting a value from 1 to 7 will stop the execution and engage one of the 6 power saving modes. Value 4 or larger allows for data memory to be erased. Value 7 allows for general purpose registers to be erased. Further details of power saving modes are implementation-defined. An interrupt automatically sets the value to 0.

IF - Interrupt flag. When set to 1 interrupts are enabled. When set to 0 interrupts are disabled.

The default value of the Control Register is 0.

When an interrupt is received, the oldest 4 bits (ES and IF) are set to 0, resuming execution and turning off interrupts.